

African Institute for Mathematical Sciences - Senegal

Academic Program (2025-2026)

Syllabus Skill Courses: **Applied Generative Agentic AI**
LLMs | Agentic Workflows | Multimodal AI

Skill Block Course

May 18 – June 10

GAAI-AIMS: Applied Generative Agentic AI

Dr. Papa Segá WADE

Contents

1	GAAI-AIMS — Applied Generative Agentic AI: LLMs, Agentic Workflows	2
1.1	1. Teaching Philosophy	2
1.1.1	1.1 Guiding Principle: Bridging SOTA Engineering and African Real-World Context	2
1.1.2	1.2 Realistic Prerequisites	3
1.1.3	1.3 Pre-course Preparation (Flipped Classroom)	3
1.1.4	1.4 Diagnostic Quiz (Session 1, 15 min)	3
1.1.5	1.5 GitHub Classroom	3
1.2	2. Course Architecture	3
1.3	3. Syllabus	4
1.3.1	MODULE 0: From Neurons to Transformers (4h)	4
1.3.2	MODULE 1: Understanding LLMs (4h)	5
1.3.3	MODULE 2: Using and Adapting an LLM (8h)	7
1.3.4	MODULE 3: LLM Agents and Autonomous Systems (6h)	10
1.3.5	MODULE 4: Evaluation, Security, and Project (8h)	12
1.4	4. Overall Assessment	13
1.5	5. Final Project: Real-World Impact	14
1.6	6. Technical Resources	14
1.6.1	Infrastructure and GPU Access	14
1.6.2	Python Libraries	15
1.6.3	Recommended Pedagogical Resources	15
1.6.4	8. Overview: 13-Session Schedule (30 Hours)	16
1.7	Appendix A: Mathematical Formalization of the Transformer	17
1.7.1	A.1 Self-Attention	17
1.7.2	A.2 Multi-Head Attention	18
1.7.3	A.3 Positional Encoding	18
1.7.4	A.4 Exercise: Manual Attention Computation	18
1.8	Appendix B: BPE Tokenization (Byte-Pair Encoding)	18
1.8.1	Principle	18
1.8.2	Why It Matters	19
1.8.3	Resource	19
1.9	Appendix C: Glossary	19

1 GAAI-AIMS — Applied Generative Agentic AI: LLMs, Agentic Workflows

Institution: AIMS (African Institute for Mathematical Sciences), Mbour, Senegal

Course Load: 30 hours (Mixed format: 3h, 2h, and 1h sessions)

Academic Year: 2025-2026

Period: May 18 – June 10 (Skill Block)

Target Audience: Master II students with heterogeneous backgrounds (Computer Science, Mathematics, Physics)

Language of Instruction: English

Lecturer bio:

Dr. Papa Séga WADE holds a Ph.D. in Computer Science (IMT Atlantique / Orange Innovation) and is an alumnus of École Polytechnique and Télécom Paris. As an Expert AI Engineer (LLM & Speech) and Fullstack Developer, he serves as the bridge between fundamental research and *Real World Impact* in Africa. His expertise spans the entire GenAI pipeline: from designing *State-of-the-Art* (SOTA) architectures to deploying *production-ready* voice and text systems tailored for African *Low-Resource Languages*. Beyond pure engineering, he approaches AI as a massive lever for digital inclusion, educational development, and healthcare innovation. A thinker, builder, and thought leader on impactful AI, he documents his vision, research, and technical implementations on his blog: papasegawade.com.

1.1 1. Teaching Philosophy

1.1.1 1.1 Guiding Principle: Bridging SOTA Engineering and African Real-World Context

This syllabus is designed for future AI architects capable of deploying resilient systems under infrastructure constraints. A physicist does not have the same instincts as a computer scientist when facing code, and a mathematician does not have the same reference points as a physicist when facing an API. The course adopts a “high floor, open ceiling” approach:

- Each session has a baseline objective (accessible to all profiles) and optional extensions (for those who progress faster)
- Concepts are first introduced through intuition and visualization before mathematical formalization
- Labs use pre-filled Colab notebooks with cells to complete, not blank scripts
- Technical vocabulary is systematically defined at first use

- Mathematical details (attention formulas, derivations) are deferred to Appendix A and used in supporting materials, not presented on the whiteboard
- Advanced topics (MCP, A2A, complex multi-agent patterns) are mentioned for awareness and available as resources for advanced profiles, without formal development during sessions

1.1.2 1.2 Realistic Prerequisites

What we assume is known	What we do NOT assume
Basic Python (variables, loops, functions)	PyTorch, TensorFlow
Linear algebra (matrices, vectors, dot product)	Neural network architectures
Elementary probability (normal distribution, Bayes)	NLP, tokenization, embeddings
Notion of gradient and optimization	Training deep learning models

1.1.3 1.3 Pre-course Preparation (Flipped Classroom)

To reduce the cognitive load during the first sessions and establish a common baseline, students are required to review the following material one week before the course begins:

- Baseline Requirement (Under 10 min): Watch this concise overview by **3Blue1Brown** to grasp the fundamental intuition of what a Large Language Model actually is before we dive into the architecture: [What is an LLM?](#)
- Going Further (Advanced): For those who want to anticipate the engineering challenges and deeply understand the fundamentals of tokenization, watch **Andrej Karpathy's** masterclass: [Let's build the GPT Tokenizer](#)

1.1.4 1.4 Diagnostic Quiz (Session 1, 15 min)

A short anonymous diagnostic quiz at the beginning of the course will cover the foundational concepts related to this module. This allows us to calibrate the final pacing and technical depth according to the cohort's actual baseline.

1.1.5 1.5 GitHub Classroom

For assignments and lab submissions, students will use a dedicated GitHub Classroom named **PSW-Generative-and-Agentive-AI-AIMS-classroom**. The instructor will add you to the classroom, or you can request to join immediately using this link: [GAAL-AIMS-Classroom](#)

1.2 2. Course Architecture

Module	Title	Hours	Sessions	Philosophy
M0	From Neurons to Transformers	4h	S1–S2	Build intuition, level the foundations
M1	Understanding LLMs	4h	S3–S4	Generative models and the LLM pipeline
M2	Using and Adapting an LLM	8h	S5–S8	From prompting to fine-tuning through RAG
M3	LLM Agents and Autonomous Systems	6h	S9–S11	Building systems that act
M4	Evaluation, Security, and Project	8h	S12–S15	Safety, evaluation, final project

Total: 30h, with approximately 40% hands-on labs integrated into each session.

1.3 3. Syllabus

1.3.1 MODULE 0: From Neurons to Transformers (4h)

Goal: Give all students, regardless of their background, the keys to understanding the Transformer. We are not repeating a deep learning course. We build a coherent narrative that leads from linear regression to attention mechanism. Detailed formulas (self-attention, multi-head) are in Appendix A. Here we build intuition.

Session 1: The Road to the Transformer: From Neurons to Attention (2h)

Baseline Objective: Understand why we moved from simple networks to Transformers, and what each architecture contributes.

Content:

Diagnostic quiz (15 min).

Progression from perceptron to MLP then CNN, each architecture adds an inductive bias (locality for CNN, sequentiality for RNN). The sequence problem and RNNs. Vanishing gradients (visualization, not formal proof).

LSTM: gates as a solution, using the analogy of a notebook with a pen, eraser, and re-reading.

Bahdanau attention: instead of memorizing everything into a fixed vector, we “look back” at each step.

Introduction to Git/GitHub (15 min): the basics for pair work, covering clone, commit, push, pull (projects will be submitted via a repo).

Lab: run a pre-trained LSTM on character-level text generation (Karpathy style), observe its limitations.

Extension for advanced profiles: Implement a simple attention mechanism (dot product) on the same example and qualitatively compare with the LSTM.

Key Resource: [The Illustrated Transformer by Jay Alammar](#), to read before or after the session.

Session 2: The Transformer: The Idea, the Architecture, the Impact (2h)

Baseline Objective: Understand the Transformer well enough to read an architecture diagram of GPT or BERT, without deriving every formula.

Content:

The fundamental idea: an attention mechanism that allows each word to “look at” all others in parallel (self-attention). Intuitive explanation of Queries, Keys, Values as an internal search engine (the full formula is in Appendix A).

Multi-head attention: why looking from multiple “angles” at once matters, illustrated with attention head visualizations.

Positional encoding: the Transformer has no notion of order, so we inject it.

High-level assembly: the complete encoder-decoder diagram.

The three families and their uses: encoder-only (BERT, for classification and NER), decoder-only (GPT, for generation), encoder-decoder (T5, for translation).

Why it changed everything: parallelization vs sequential RNN, scaling capabilities.

Lab: load a small pre-trained model via Hugging Face using unsloth, visualize attention weights on a sentence, observe how different heads capture different relationships.

Extension for advanced profiles: Manually compute attention on a 3-word x 3-word example with given Q, K, V matrices (exercise in Appendix A).

Key Resource: [Attention Is All You Need, annotated version \(Harvard NLP\)](#)

1.3.2 MODULE 1: Understanding LLMs (4h)

Goal: Understand what an LLM is, how it is trained, and what models are available. Here we move from “I understand the Transformer” to “I understand GPT|Llama|Qwen|Kimi|Mistral.” This module has been condensed: overly detailed concepts (step-by-step BPE|WordPiece|SentencePiece, in-depth RLHF|DP0) are mentioned but not formally developed. The essential point is understanding the overall pipeline.

Session 3: Panorama of Generative AI and the LLM Pipeline (2h)

Baseline Objective: Situate LLMs within the landscape of generative models and understand the main stages of creating an LLM.

Content:

Part One (50 min): Generative AI Panorama.

What does “generative” mean: $P(x)$ vs $P(y|x)$. The four major families presented through intuition, one slide each, no formulas:

- **Autoregressive (GPT):** predict the next word, one at a time. This is what today's LLMs are.
- **VAE:** compress and reconstruct. Mentioned as a historical approach.
- **GAN:** generator vs discriminator. Mentioned as a historical approach.
- **Diffusion:** noise to image. Mention Stable Diffusion and DALL-E as concrete examples.

Key message: LLMs are autoregressive models. The other families exist but are not the focus of this course.

Multimodal vision-language: brief overview of CLIP, GPT-4V, Gemini, LLaVA (show an example of an image-to-text prompt to make it concrete).

Note: Speech technologies (Whisper, MMS, Qwen-ASR) are covered in Session 4 as part of the open models and multimodal panorama. This frees up time in S3 and allows presenting Speech in a more concrete context, after students have understood what an LLM is.

Part Two (40 min): The LLM Pipeline, the essentials.

The three key stages, presented as a story:

1. **Pre-training:** predicting the next word on billions of tokens. Subword tokenization: the principle in 2 minutes (words are split into sub-words. The detailed BPE algorithm is in Appendix B).
2. **Instruction-tuning (SFT):** transforming a model that “completes text” into one that “answers questions.” Show a concrete before/after example.
3. **Alignment:** the model must refuse dangerous requests and follow human intentions. Mention RLHF and DPO as the two main approaches (DPO being the dominant simplification in 2025-2026), without going into technical details. One slide is sufficient.

Lab (30 min): Explore the Hugging Face Hub (search models by task, read a model card), tokenize a sentence in French and English with `tiktoken` (compare token counts, which gives intuition about the model's “worldview” and the cost premium for non-English languages).

Extension for advanced profiles: Read Section 4 of the [original DPO paper](#) to understand the mathematical derivation.

Key Resources: [LLM Course](#), [Hugging Face](#) | [nanoGPT \(Andrej Karpathy\)](#) | [Unsloth library](#)

Session 4: Open Models, Costs, Reasoning, and Multimodal Speech (2h)

Baseline Objective: Know how to choose a model suited to a need, understand real costs, and discover the multimodal Speech dimension, which is strategic for Africa.

Content:

Part One (1h): Choosing and Sizing an LLM.

Open vs closed: what does “open source” mean for an LLM? (open weights open data free license).

Major model families: GPT-4/o (OpenAI, closed), Llama 3 (Meta, open), Mistral (Europe, open), Gemma (Google, open).

Selection criteria: performance, cost, license, size, local deployment vs cloud.

The real cost of an LLM: input/output tokens, price per million tokens, GPU cost for local inference.

Quantization (INT8, INT4, GGUF): reducing size to run locally.

Reasoning models: OpenAI o1/o3, DeepSeek-R1, Gemini “thinking,” with internal chain-of-thought. More compute at inference time means better complex answers (math, code, logic).

Context window: from 8K to 1M+ tokens in 2026, how to handle long documents, when to split, strategies (map-reduce, sliding window, RAG).

Frugal AI and African context: infrastructure constraints (bandwidth, GPU, cost), compact and quantized models, African languages. We will use the **LiteRT-LM** from Google-DeepMind.

Part Two (30 min): Multimodal Speech, why it is strategic in Africa.

- **Qwen3-ASR** (Alibaba): state-of-the-art open speech recognition model with excellent multilingual capabilities, more than 40 millions of hours speech data pretrained.
- **Whisper** (OpenAI): speech recognition for 99 languages, the current standard.
- **MMS** (Meta): ASR and TTS for **1,100+ languages** including many African languages (Wolof, Swahili, Yoruba, Hausa, and more).
- **SeamlessM4T** (Meta): speech-to-speech and speech-to-text translation for 100 languages.

Why this is strategic in Africa: oral tradition, varying literacy rates, digital inclusion through voice interfaces in local languages (agriculture, health, public administration). Open-source Speech models offer a unique opportunity to build accessible applications without depending on proprietary English-centric solutions.

Lab (30 min): Load a quantized model with **bitsandbytes** or **Unsloth**, compare speed and quality, test an explicit chain-of-thought prompt vs a standard prompt.

Extension for advanced profiles: Calculate the theoretical memory cost of a 7B model in FP16 vs INT4, verify against real figures. Test Whisper on an audio sample in a local language if available.

Key Resources: [Google DeepMind AI Research Foundations](#) | [MMS \(Meta\)](#) | [SeamlessM4T \(Meta\)](#) | [Qwen3-ASR](#)

1.3.3 MODULE 2: Using and Adapting an LLM (8h)

Goal: Move from understanding to practice. Know how to use an LLM effectively (prompting), connect it to external knowledge (RAG), and adapt it to a domain (fine-tuning).

Session 5: Prompt Engineering and Context Management (2h) **Baseline Objective:** Master prompting techniques to obtain reliable and reproducible results from an LLM.

Content:

Why prompting is a key skill: same model, different prompt, radically different result.

Fundamental techniques: zero-shot, few-shot, role, format (JSON, table, bullet points).

Advanced techniques: chain-of-thought (“reason step by step”), self-consistency (vote across multiple responses).

Context window management: the limits (8K to 1M tokens), how to structure a long prompt, strategic placement of information (beginning/end vs lost in the middle), when to summarize vs split.

Limits of prompting: hallucinations (the model makes things up), knowledge cutoff, context window. This naturally leads to RAG and fine-tuning.

Lab: design prompts for 4 AIMS tasks (summarizing a scientific article, explaining a mathematical concept, generating Python code, extracting structured information from a French text), compare zero-shot vs few-shot vs chain-of-thought on each task.

Extension for advanced profiles: Implement a self-consistency pipeline with majority voting over 5 responses, measure improvement on a set of math questions.

Key Resource: [Prompt Engineering Guide](#), comprehensive reference for all techniques.

Session 6: RAG: Retrieval-Augmented Generation (2h) **Baseline Objective:** Understand and implement a system that allows an LLM to answer questions based on specific documents.

Content:

The problem: the LLM does not know your internal documents.

The RAG idea: chunks, embeddings, vector store, retrieval, LLM, response.

Key components: embeddings (Sentence-Transformers, dense vectors that capture meaning), vector stores (Chroma), intelligent chunking (by paragraph, by tokens, with overlap).

Dense vs sparse search and reranking (mentioned, not developed. The essential thing is a functional pipeline).

Lab (1h): build a mini-RAG from scratch on an AIMS corpus. Load a PDF, split it, create embeddings, store in Chroma, ask questions with and without RAG.

Extension for advanced profiles: Implement a reranker with a cross-encoder and measure the impact on result quality. Experiment with different chunking strategies.

Key Resource: [RAG from Scratch Tutorial](#), [Hugging Face Blog](#)

Session 7: Lightweight Fine-tuning with LoRA/QLoRA (2h) **Baseline Objective:** Understand when and how to adapt an LLM to a specific domain with limited resources.

Content:

Decision tree: if prompting is enough, use prompting. If you need specific knowledge, use RAG. If you need to change style or behavior, use fine-tuning.

The problem with full fine-tuning (GPU cost).

LoRA: adding small low-rank matrices to certain layers, freezing the rest, resulting in 99% fewer trainable parameters.

QLoRA: the base model is quantized to 4 bits, so a 24 GB GPU is sufficient for a 70B model.

Data preparation: instruction-response format, quality over quantity.

Synthetic data generation: when datasets do not exist (a frequent case for African languages), use an LLM as a “teacher” to generate instruction-response pairs.

Techniques: Self-Instruct (starting from 175 seed examples), distillation from a strong model (e.g., Llama 3 70B to generate a dataset for fine-tuning a 7B), Evol-Instruct (progressive complexification), quality filtering (IFD, LLM-as-a-Judge).

Lab: fine-tune a small model (Llama 3.2 1B or Phi-3 mini) using Hugging Face PEFT and **Unsloth** on a QA dataset, compare before and after.

Note on Unsloth: **Unsloth** accelerates LoRA/QLoRA fine-tuning by 2-5x with 80% less memory. It patches the model’s attention and MLP layers with custom CUDA kernels and is fully compatible with the Hugging Face ecosystem (Trainer, PEFT, TRL). In the lab, students will use Unsloth’s `FastLanguageModel` to load a quantized model and fine-tune it efficiently, even on a free Colab T4 GPU.

Extension for advanced profiles: Experiment with different LoRA hyperparameters (rank r , alpha, dropout) and measure the impact. Analyze the learned weights.

Key Resources: [Practical LoRA Guide 2025](#) | [Unsloth GitHub](#) | [Synthetic Data Guide \(Prem AI\)](#) | [Distilabel \(Argilla/Hugging Face\)](#)

Session 8: Evaluating an LLM Pipeline + Project Launch (2h) **Baseline Objective:** Know how to rigorously evaluate the outputs of an LLM. Choose and launch the final project.

Content:

Part One (1h): Evaluation. Why it is hard: answers are not binary. Classic metrics: exact match, F1, BLEU, ROUGE, perplexity (and the limits of each).

LLM-as-a-Judge: using an LLM to evaluate responses (advantage: scalable. Risk: judge bias).

Short lab: evaluate the mini-RAG from Session 6 with 10 questions, implement a simple LLM-as-a-Judge.

Part Two (1h): Project launch. Presentation of project topics (see Section 5). Formation of pairs/trios (mixed profiles). Topic selection. Writing the functional specification: objective, tools, inputs/outputs, guardrails, success criteria.

Extension for advanced profiles: Implement an evaluation with the RAGAS framework.

Key Resource: [RAGAS, RAG evaluation framework](#)

1.3.4 MODULE 3: LLM Agents and Autonomous Systems (6h)

Goal: Move from the LLM as a “chatbot” to the LLM as an “agent” capable of acting, using tools, and solving complex problems. This module is calibrated for a first encounter: we build an agent, we understand memory and workflows, and we mention emerging protocols (MCP, A2A) as general knowledge without formal development.

Session 9: Introduction to LLM Agents (2h) **Baseline Objective:** Understand what an LLM agent is, how it works, and build a first simple agent.

Content:

From chatbot to agent: an agent uses tools, maintains memory, and plans multi-step actions.

Anatomy: brain (LLM), tools (function calling), memory, Observe, Reflect, Act loop (ReAct pattern).

Function calling and tool use: the model generates a structured JSON to call a function.

Framework overview: LangChain/LangGraph (most widely used, graph-based), CrewAI (role-based agents), GoogleADK (Google, conversational).

Lab: build a LangChain agent with a calculator and Wikipedia search, observe the agent’s “thinking” and how it decides which tool to use.

Extension for advanced profiles: Create a custom tool (symbolic equation solver with SymPy) and integrate it into the agent. Analyze failure cases of the agentic loop.

Key Resource: [Berkeley CS 294, LLM Agents](#), reference syllabus.

Session 10: Memory, Workflows, and Protocol Awareness (2h) **Baseline Objective:** Understand agent memory systems and build a multi-step workflow with LangGraph.

Content:

Part One (45 min): Agent Memory.

Types of memory (presented as an analogy with human memory):

- **Conversation** (short-term): the chat history.
- **Summary** (medium-term): summarizing past exchanges when the history becomes too long.
- **Vectorial** (long-term): this is RAG on the conversation history. Messages are stored as embeddings to retrieve relevant information.
- **External** (databases, files): the agent can read and write to files or a database.

Part Two (45 min): Workflows with LangGraph.

Modeling a workflow as a graph of nodes and edges: each node is a step (search, analysis, writing), edges define transitions (with conditional branching and human validation). Demo: a “search, verify, format response” workflow.

Part Three (15 min): Awareness, Interoperability Protocols.

MCP and A2A are presented as technological awareness, not as skills to master. The goal is for students to know these standards exist and understand the problem they solve.

- **MCP (Model Context Protocol):** Anthropic’s standard that became widely adopted in 2026. The idea in one sentence: instead of connecting each agent to each tool ($M \times N$ integrations), MCP defines a common protocol ($M + N$). Client-server architecture with 3 primitives: tools, resources, prompts. Show one diagram.
- **A2A (Agent-to-Agent):** Google/Linux Foundation protocol enabling agents from different frameworks to communicate. Mention its existence, do not develop.

Lab (15 min): Add conversational memory to the agent from Session 9. Build a mini LangGraph workflow (search, verify, respond).

Extension for advanced profiles: Implement a minimal MCP server that exposes a simple tool (reading a CSV file), connect it to an MCP client. Resources: [Introduction to MCP \(Anthropic\)](#) | [Complete MCP Guide \(Machine Learning Mastery\)](#)

Key Resource: [LangGraph Documentation](#)

Session 11: Multi-Agent Patterns and Project Progress (2h) **Baseline Objective:** Know the two most common multi-agent patterns. Make progress on the project prototype.

Content:

Part One (30 min): Multi-Agents, the essentials.

Why multiple agents? When a task is too complex for a single agent, we decompose it.

The **2 fundamental patterns** (developed):

- **Orchestrator:** a “manager” agent that receives the task, decides which specialized agent to call, and synthesizes results. AIMS example: a tutor agent that delegates to a “math” agent and a “research” agent.
- **Pipeline:** A, then B, then C. Each agent handles one step then passes to the next. AIMS example: research, then writing, then fact-checking.

Other patterns (mentioned, not developed):

- *Debate:* two agents argue, a third arbitrates. Useful but complex to orchestrate.

- *Hierarchy*: supervisor plus specialists. A variant of the orchestrator with more levels.

Designing an agent: functional specification (objective, tools, inputs/outputs, guardrails, evaluation).

Part Two (1h30): Project work.

Pair/trio work on the prototype with guidance from the instructor and the tutor. Individual technical check-in per group (5 min each).

Extension for advanced profiles: Implement a debate pattern between two agents (one “for,” one “against”) with a judge agent.

Key Resource: [CrewAI vs LangGraph vs AutoGen \(DataCamp\)](#)

1.3.5 MODULE 4: Evaluation, Security, and Project (8h)

Goal: The crucial dimensions that technical courses often neglect (security, ethics, bias) and project finalization. This module is reinforced compared to V0 (8h instead of 6h) to allow realistic project time.

Session 12: Security, Bias, and Responsible AI (2h) **Baseline Objective:** Know the main risks of LLM systems and know how to set up guardrails.

Content:

Hallucinations: why they occur, how to detect them, how to reduce them (RAG, verification, prompting).

Adversarial attacks: prompt injection (injecting malicious instructions), jailbreak (bypassing guardrails), data exfiltration.

Bias and fairness: LLMs reproduce and amplify biases from training data, with specific stakes for the African context (languages, cultures, underrepresented perspectives).

Governance and regulation: EU AI Act (main principles), model cards, environmental impact (carbon cost).

Lab: attack/defense workshop. Attempt injections on a chatbot, implement simple guardrails (input filtering, robust system prompt), test a model’s biases (ask the same questions while changing geographic/gender context).

Extension for advanced profiles: Write a complete model card for the model fine-tuned in Session 7, following the template from [Mitchell et al. \(2019\)](#).

Key Resource: [OWASP Top 10 for LLM Applications](#)

Session 13: Supervised Project Work (1/2) (2h) **Objective:** Dedicated session for prototype development.

Content: Group debrief (15 min): feedback on difficulties encountered, sharing solutions between groups. Pair/trio work (1h30): prototype development, integration of security

guardrails (from S12), testing. Informal mini-demo (15 min): each group shows their progress in 2 minutes.

Session 14: Supervised Project Work (2/2) + Peer Review (2h) **Objective:** Finalize the prototype and the report.

Content: Pair/trio work (1h15): prototype finalization, report writing. Peer review (45 min): each group presents for 10 minutes to another group, structured cross-feedback (a form: “strength / area for improvement / question”).

Deliverables expected for Session 15:

- Functional prototype (notebook or script in a GitHub repo classroom)
- Short PDF report (4-6 pages): problem, architecture, results, limitations, perspectives
- Presentation slides

Session 15: Final Project Presentations (2h) **Format:** 10-minute presentation (problem, approach, live demo, results) + 5-minute Q&A (instructor + tutor) + 2-minute oral feedback per group.

Evaluation Criteria:

Criterion	Weight
Relevance of the use case	15%
Technical rigor	30%
Quality of evaluation	20%
Ethical and security considerations	5%
Presentation quality	20%
Written report	10%

1.4 4. Overall Assessment

Component	Weight	Detail
Graded Labs	15%	3 graded labs: S6-RAG, S7-LoRA, S9-Agent. Notebooks / Python script submitted via GitHub
Midterm Assessment	25%	MCQ + short open questions after S4, covering M0 and M1. No trick questions: assess understanding, not memorization.
Final Project	60%	Report + prototype + presentation (rubric above)

1.5 5. Final Project: Real-World Impact

The final project is the culmination of this course. Rather than a theoretical exercise, you will build and deploy a functional, production-ready GenAI system addressing a concrete challenge.

Specific project topics and technical specifications will be unveiled during **Session 8** to ensure you have the necessary architectural foundations to choose wisely. The projects will directly target key sectors for African development, including:

- **Healthcare & Triage** : Designing autonomous agents with strict guardrails based on official medical documentation.
- **Education & STEM** : Building adaptive tutoring systems capable of scaling personalized learning.
- **Digital Inclusion** : Creating multimodal workflows (Speech/Text) adapted to local low-resource languages.
- **Data Intelligence** : Developing autonomous pipelines for telecom or agricultural data analysis.

Students will also have the opportunity to propose their own impact-driven topics, subject to instructor validation.

1.6 6. Technical Resources

1.6.1 Infrastructure and GPU Access

Tool	Usage	Access	Note
Google Colab	Notebooks with GPU (T4/A100)	Web browser	Plan Colab Pro credits for the cohort. The free tier is insufficient for 20+ simultaneous students. Contact Google Education or AWS Academy.
Ollama/LiteRT-LM	Local model execution	Installation on local machine	Backup if internet is unstable, models pre-downloaded on USB drives
Python 3.10+	Primary language	Pre-installed on Colab	
Git/GitHub	Versioning and project submission	Free GitHub account	Crash course in S1 (15 min)

Network contingency plan: all notebooks are downloadable for offline use. Ollama models (Llama 3.2 1B, Phi-3 mini) are pre-downloaded on local machines or USB drives. Labs are designed to work with 1-3B parameter models, compatible with 8 GB of RAM.

1.6.2 Python Libraries

Library	Usage
<code>transformers</code> (Hugging Face)	Model loading and inference
<code>peft</code>	LoRA/QLoRA fine-tuning
<code>unsloth</code>	Accelerated fine-tuning (2-5x faster, 80% less memory)
<code>datasets</code>	Data loading
<code>sentence-transformers</code>	Embeddings for RAG
<code>chromadb</code>	Vector database for RAG
<code>langchain</code> + <code>langgraph</code>	Agents and workflows
<code>bitsandbytes</code>	Quantization
<code>tiktoken</code>	Tokenization
<code>distilabel</code>	Synthetic data generation

1.6.3 Recommended Pedagogical Resources

Resource	Type	Link
Hugging Face LLM Course	Free online course	huggingface.co/learn/llm-course
The Illustrated Transformer	Visual article	jalammar.github.io
Attention Is All You Need (annotated)	Annotated paper	nlp.seas.harvard.edu
nanoGPT (Karpathy)	Code + video	github.com/karpathy/nanoGPT
RAG from Scratch (HF Blog)	Tutorial	huggingface.co/blog/make-your-own-rag
Practical LoRA Guide	Tutorial	amirteymoori.com
Unsloth Fine-tuning	Library + notebooks	github.com/unslothai/unsloth
Synthetic Data for Fine-tuning	Complete guide	blog.prem.ai
MMS (Meta Massively Multilingual Speech)	Speech model, 1100+ languages	huggingface.co/facebook/mms-tts
SeamlessM4T (Meta)	Multilingual speech translation	ai.meta.com/seamless
Intro to MCP (Anthropic)	Course	anthropic.skilljar.com
Complete MCP Guide	Article	machinelearningmastery.com

Resource	Type	Link
CrewAI vs LangGraph vs AutoGen	Comparison	datacamp.com/tutorial
Google DeepMind AI Research Foundations	Complete curriculum	skills.google/paths/3135
Berkeley CS 294 LLM Agents	Reference syllabus	rdi.berkeley.edu/llm-agents
OWASP Top 10 for LLM	Security	owasp.org
Prompt Engineering Guide	Reference	promptingguide.ai
Context Window Management	Practical guide	oneuptime.com/blog

1.6.4 8. Overview: 13-Session Schedule (30 Hours)

Week 1: Foundations & SOTA Engineering (3h Blocks)

The objective is to absorb all the technical complexity before the break week.

Day	Date	Dur.	Daily Theme	Pedagogical Flow
D1	May 18	3h	M0: From Neurons to Attention (Covers S1)	Quiz. RNN/LSTM/Attention theory. Massive Git setup. Extended LSTM Lab.
D2	May 19	3h	M0/M1: Architecture & Pipeline (Covers S2 + S3 part 1)	The Transformer from A to Z. GenAI Panorama. Lab: Attention Visualization.
D3	May 20	3h	M1: Frugality & Multimodality (Covers S3 part 2 + S4)	LLM Pipeline, Open Source Models, and total focus on Speech (MMS, Qwen3-ASR) for Africa. Midterm Assessment (last 30 min).
D4	May 21	3h	M2: Context Mastery (Prompt & RAG) (Covers S5 + S6)	Context limits. Complete RAG architecture. Intensive Lab: RAG from scratch on Chroma.
D5	May 22	3h	M2: Fine-Tuning & Launch (Covers S7 + S8 part 2)	QLoRA with Unsloth. Synthetic data generation. Presentation of project topics. Team formation.

(Break Week: May 25–29.)

Week 2 & 3: Agentic Workflows & Deployment (2h/1h Blocks)

Upon return, the focus shifts entirely to multi-agents and project execution.

Day	Date	Dur.	Daily Theme	Pedagogical Flow
D6	Jun 01	2h	M2/M4: Evaluation & Security (Covers S8 part 1 + S12)	LLM-as-a-Judge, RAGAS, Hallucinations, and Security (AI Act, Bias). Locking down guardrails before coding.
D7	Jun 02	2h	M3: Agentic Intelligence (Covers S9)	From completion to action. The ReAct pattern. Lab: LangChain + Tools.
D8	Jun 03	2h	M3: Workflows & Memory (Covers S10)	LangGraph. Introduction to MCP/A2A. Lab: Conditional workflow.
D9	Jun 04	2h	M3: Multi-Agents & Hackathon 1 (Covers S11 + S13 part 1)	Advanced patterns (Orchestrator, Pipeline). Start of supervised Hackathon on projects.
D10	Jun 05	2h	M4: Hackathon 2 (Covers S13 part 2)	Pure code. Architecture debugging.
D11	Jun 08	2h	M4: Hackathon 3 (Covers S14 part 1)	Finalizing prototypes.
D12	Jun 09	1h	M4: Code Audit & Peer Review (Covers S14 part 2)	Cross-group code review. Preparing reports.
D13	Jun 10	2h	M4: Defense (Covers S15)	Final defenses and live demonstrations.

1.7 Appendix A: Mathematical Formalization of the Transformer

This appendix is intended for mathematical and physics profiles who wish to understand the formal foundations. It is not covered in lectures but is used in notebooks and available as reference.

1.7.1 A.1 Self-Attention

Given an input sequence X of dimension (n, d) , we compute:

- Queries: $Q = XW_Q$

- Keys: $K = XW_K$
- Values: $V = XW_V$

Attention is defined by:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The factor $\sqrt{d_k}$ prevents the dot products from growing too large as the dimension increases.

1.7.2 A.2 Multi-Head Attention

We apply h attention heads in parallel, each with its own projection matrices, then concatenate:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

1.7.3 A.3 Positional Encoding

Sinusoidal encoding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

1.7.4 A.4 Exercise: Manual Attention Computation

Given a sequence of 3 tokens with $d = 4$ and matrices Q, K, V provided, compute step by step: QK^T , the division by $\sqrt{d}$, row-wise softmax, then the final product.

(Numerical matrices to be provided in the S2 lab notebook.)

1.8 Appendix B: BPE Tokenization (Byte-Pair Encoding)

Reference for the curious. Mentioned in S3, not developed in lectures.

1.8.1 Principle

BPE is a compression algorithm adapted for tokenization:

1. Start with a vocabulary of all individual characters
2. Count all adjacent token pairs in the corpus
3. Merge the most frequent pair into a new token
4. Repeat until the desired vocabulary size is reached

1.8.2 Why It Matters

- A rare word is never “unknown.” It is split into known sub-words
- Agglutinative languages (Wolof, Swahili) can be better handled
- Token count varies by language: a French sentence often produces 30-50% more tokens than English, resulting in higher cost

1.8.3 Resource

[BPE Tutorial](#), [Hugging Face NLP Course](#)

1.9 Appendix C: Glossary

Term	Definition
Attention	A mechanism that allows a model to weight the importance of each element in a sequence relative to the others
Agent	A system that uses an LLM as its “brain” to plan actions, use tools, and accomplish tasks autonomously
BPE	Byte-Pair Encoding: a tokenization algorithm that splits words into frequent sub-words
Chain-of-thought	A prompting technique that asks the model to reason step by step before answering
Chunking	Splitting a document into fixed-size or semantic segments for RAG
Context window	The maximum number of tokens an LLM can process in a single request (input + output)
DPO	Direct Preference Optimization: an alignment method that learns directly from human preferences without a separate reward model
Embedding	A dense vector representation of a word, sentence, or document in a continuous space
Fine-tuning	Adapting a pre-trained model to a specific task or domain
Function calling	An LLM’s ability to generate a structured function call (JSON) to interact with external tools
Hallucination	Generation of fictitious content presented as factual by an LLM
LoRA	Low-Rank Adaptation: a fine-tuning method that only trains small matrices added to the model’s layers
LLM	Large Language Model: a large-scale language model, pre-trained on vast text corpora
MCP	Model Context Protocol: an interoperability standard allowing an agent to connect to tools and data sources via a unified protocol
A2A	Agent-to-Agent: a Google/Linux Foundation protocol for inter-agent communication
Prompt	The input text provided to an LLM to guide its response
QLoRA	LoRA applied to a model quantized to 4 bits, drastically reducing memory requirements

Term	Definition
Quantization	Reducing the numerical precision of a model's weights (e.g., FP16 to INT4) to reduce size and speed up inference
RAG	Retrieval-Augmented Generation: an architecture combining document retrieval and generation to answer questions from documents
ReAct	An agentic pattern alternating Reasoning and Acting
RLHF	Reinforcement Learning from Human Feedback: an alignment method using a reward model trained on human preferences
SFT	Supervised Fine-Tuning: supervised training on instruction-response pairs
Token	The basic unit processed by an LLM, which can be a word, sub-word, or character
Transformer	A neural network architecture based on attention, without recurrence, forming the foundation of all modern LLMs
Unsloth	An optimization library that accelerates LoRA/QLoRA fine-tuning with custom CUDA kernels, reducing time and memory usage
Vector store	A database specialized in storing and searching embedding vectors (e.g., Chroma, FAISS)

*Syllabus, April 15, 2026. Calibrated for a first GenAI course following DL/ML modules.
Working document, to be iterated with the AIMS teaching team.*